Banco de Dados

Normalização — 1ª Forma Normal (1FN)

Até agora...

Já sabemos:

- transformar um **DER em tabelas**
- identificar **chaves**
- identificar **dependências funcionais**
- identificar dependências **parciais**, **transitivas** e **multivaloradas**

Hoje: por que uma tabela "correta" ainda pode estar mal organizada.

Um problema para começar

Considere uma tabela de pedidos:

pedido	cliente	produtos
1001	Ana	Mouse, Teclado
1002	João	Monitor
1003	Maria	Mouse, Teclado, Webcam

Problema: `produtos` → **vários valores em uma única célula**.

Por que isso é um problema?

O banco deveria conseguir tratar cada produto como um **valor individual**.

Mas temos:

```
Mouse, Teclado
```

> **É aqui que entra a normalização:** organizar os dados para evitar esse tipo de problema.

Normalização é o processo de organizar as relações de um banco de dados para reduzir:

- redundância
- inconsistência
- anomalias (de inserção, alteração e exclusão)
- estruturas inadequadas

As formas normais

```
1FN → 2FN → 3FN → BCNF → 4FN
```

Cada forma normal impõe uma condição a mais sobre a relação.

1ª Forma Normal: Uma relação está na **1ª Forma Normal** quando todos os seus atributos têm valores **atômicos**.

Mas o que é "atômico"?

Valor atômico

Um valor é atômico quando representa **um único valor** para aquele atributo.

matrícula	nome
101	Ana
102	João

Cada célula deve possuir um único valor.

Um exemplo que NÃO está na 1FN

matrícula	nome	telefones
101	Ana	9999-1111, 9888-2222
102	João	9777-3333

Na linha da Ana:

```
telefones = 9999-1111, 9888-2222
```

`telefones` **não** guarda um valor atômico.

Como colocar na 1FN

Uma solução: uma linha por telefone.

matrícula	telefone
101	9999-1111
101	9888-2222
102	9777-3333

Agora cada célula tem um único telefone.

Mas surge uma pergunta

matrícula	telefone
101	9999-1111
101	9888-2222
102	9777-3333

Qual é a chave dessa relação agora?

Resposta: `(matrícula, telefone)` — a chave muda quando a estrutura muda.

Outro exemplo: pedidos

pedido	produtos
1001	Mouse, Teclado
1002	Monitor
1003	Mouse, Teclado, Webcam

Mesmo problema: `produtos` guarda mais de um valor por célula.

Colocando pedidos na 1FN

pedido	produto
1001	Mouse
1001	Teclado
1002	Monitor
1003	Teclado
1003	Webcam

Um produto por linha.

Mais um exemplo: habilidades

funcionário	habilidades
Maria	Python, SQL, Excel
Beatriz	Java

Mesmo padrão, contexto diferente: sempre que uma célula acumula **vários valores do mesmo tipo**, é sinal de alerta para a 1FN.

E se fizermos assim?

matrícula	nome	telefone1	telefone2	telefone3
101	Ana	9999-1111	9888-2222	NULL
102	João	9777-3333	NULL	NULL

Cada célula tem um único valor. Está na 1FN?

Não necessariamente.

Grupos repetitivos

`telefone1`, `telefone2`, `telefone3` são colunas diferentes representando **o mesmo tipo de informação**, repetida várias vezes.

Isso também acontece com:

- `produto1`, `produto2`, `produto3`
- `email1`, `email2`, `email3`
- `habilidade1`, `habilidade2`, `habilidade3`

Isso é um **grupo repetitivo** — e também viola a 1FN.

Por que grupos repetitivos são ruins

Um aluno com 5 telefones → preciso de 5 colunas.

Um aluno com 6 telefones? → preciso criar `telefone6`.

A estrutura da tabela passa a depender da quantidade de dados.
Isso não escala.

Uma estrutura melhor

Em vez de uma tabela só:

```
ALUNO(matricula, nome, telefone1, telefone2, telefone3, ...)
```

Separamos em duas:

```
ALUNO(matricula, nome)
TELEFONE_ALUNO(matricula, telefone)
```

Resultado

ALUNO

matrícula	nome
101	Ana
102	João

TELEFONE_ALUNO

matrícula	telefone
101	9999-1111
101	9888-2222
102	9777-3333

“ Agora cabe **qualquer quantidade** de telefones. ”

Atenção: 1FN não é "quebrar tudo"

A 1FN **não** significa "uma coluna só pode ter um tipo de informação".

matrícula	nome	telefone
101	Ana	9999-1111

Isso é perfeitamente válido — uma célula, um valor.

E datas?

matrícula	nome	nascimento
101	Ana	15/03/2000

Está na 1FN? **Sim.**

`15/03/2000` representa um único valor de data.

E endereços?

matrícula	endereço
101	Rua A, 123, Centro, Itapipoca

Isso viola a 1FN? **Não necessariamente.**

A atomicidade depende do que o sistema precisa consultar.

Atomicidade depende do contexto

Se o sistema só precisa **exibir** o endereço → um único valor está ok.

Se o sistema precisa **consultar separadamente** rua, número, bairro e cidade → melhor modelar cada parte em sua própria coluna.

1FN não é "quebrar todo valor composto" — é garantir que cada valor seja único para o que o sistema precisa fazer com ele.

Vamos praticar

Exercício 1

código	nome	telefones
1	Ana	9999-1111, 9888-2222
2	João	9777-3333

Essa relação está na 1FN?

A) Sim, só existe uma coluna de telefones
B) Não, uma célula tem mais de um telefone
C) Sim, telefone é um atributo simples
D) Não, toda tabela precisa de chave estrangeira

Resposta 1

B) Não, porque uma célula possui mais de um telefone.

Na linha da Ana: `9999-1111, 8888-2222` são dois valores para o mesmo atributo.

Exercício 2

pedido	produto1	produto2	produto3
1001	Mouse	Teclado	NULL
1002	Monitor	NULL	NULL

Essa relação está na 1FN?

A) Sim, cada célula tem um valor
B) Não, existem grupos repetitivos
C) Sim, produto é uma entidade
D) Não, existem valores NULL

Resposta 2

B) Não, porque existem grupos repetitivos.

`produto1`, `produto2`, `produto3` representam várias ocorrências do mesmo atributo.

Correção:

pedido	produto
1001	Mouse
1001	Teclado
1002	Monitor

Exercício 3

```
ALUNO(
    matricula PK,
    nome,
    curso,
    telefones
)
```

Um aluno pode ter vários telefones. O que está errado?

Resposta 3

`telefones` pode guardar vários valores para o mesmo aluno — viola a atomicidade da 1FN.

Correção:

```
ALUNO(matricula PK, nome, curso)
TELEFONE_ALUNO(matricula FK, telefone)
```

Resumindo a 1FN

Para estar na 1FN, evitamos:

Múltiplos valores em uma célula

```
telefone = 9999-1111, 9888-2222
```

Grupos repetitivos

```
telefone1, telefone2, telefone3
```

Um atributo → um valor por célula. Múltiplas ocorrências vão para outra relação.

E depois da 1FN?

Estar na 1FN não significa que a tabela está perfeitamente organizada.

Ainda podem existir:

- dependências parciais
- dependências transitivas
- redundâncias e anomalias

Por isso existem outras formas normais.

Próxima aula: 2FN

O que acontece quando um atributo depende de **apenas parte** de uma chave composta?

```
(matricula, cod_disciplina) → nota
matricula → nome_aluno
```

Vamos descobrir por que isso é um problema.

Fechando

Normalização → organizar relações
1FN → valores atômicos
Grupos repetitivos → devem ser eliminados
Decomposição → representa múltiplos valores corretamente

Próxima aula: **2FN**